

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»**

(Университет ИТМО)

Факультет Прикладной информатики

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии

**ОТЧЕТ ПО Лабораторной работе 3 Проектирование и реализация баз
данных**

**Тема: «Создание таблиц базы данных PostgreSQL. Заполнение таблиц
рабочими данными»**

Обучающийся: Ипатова Ульяна Юрьевна K3239

Вариант 10

Санкт-Петербург 2026

Цель работы: овладеть практическими навыками создания таблиц базы данных PostgreSQL 1X, заполнения их рабочими данными, резервного копирования и восстановления БД.

Практическое задание:

1. Создать базу данных с использованием pgAdmin 4 согласно индивидуальному заданию.
2. Создать схему в составе базы данных.
3. Создать таблицы базы данных.
4. Установить ограничения на данные: Primary Key, Unique, Check, Foreign Key.
5. Заполнить таблицы БД рабочими данными.
6. Создать резервную копию БД.
7. Создать две резервные копии: в формате Custom для восстановления БД и в текстовом формате для листинга в отчете.
8. Выполнить восстановление БД.

Выполнение:

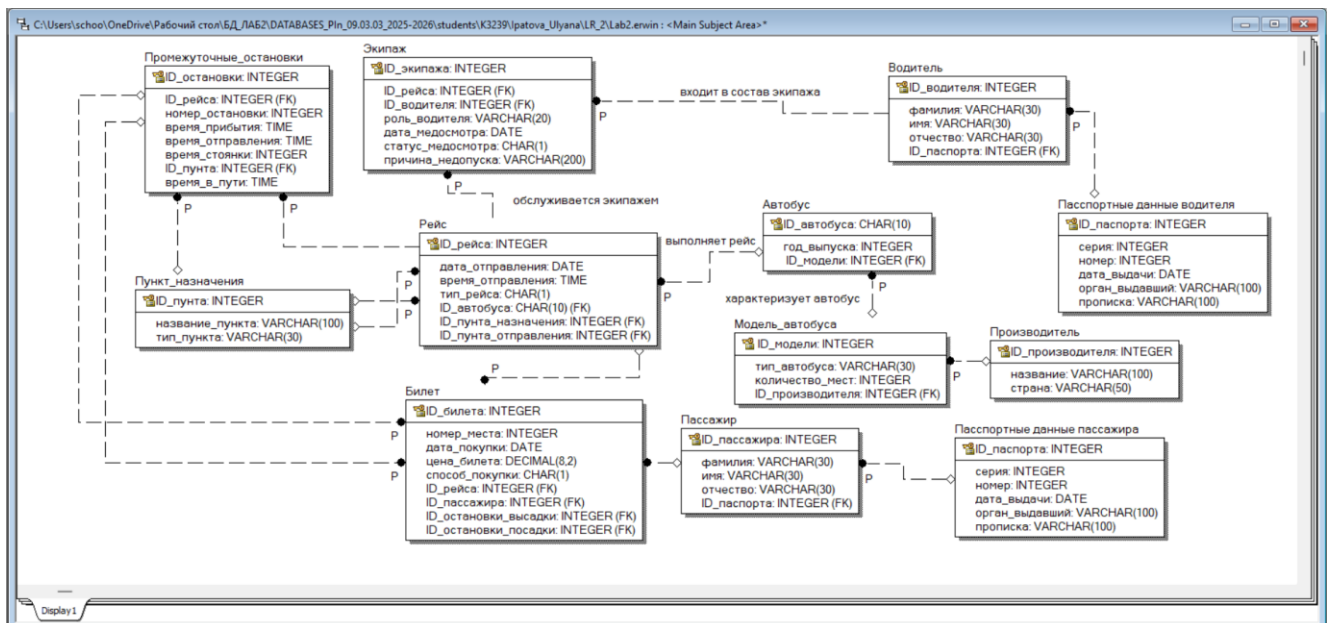
Вариант 10

Наименование базы данных: «Автовокзал».

Имя реализованной базы данных в PostgreSQL: bus_station

Имя схемы: bus_station_sch

Схема инфологической модели БД ЛР 2 (IDEF1X):



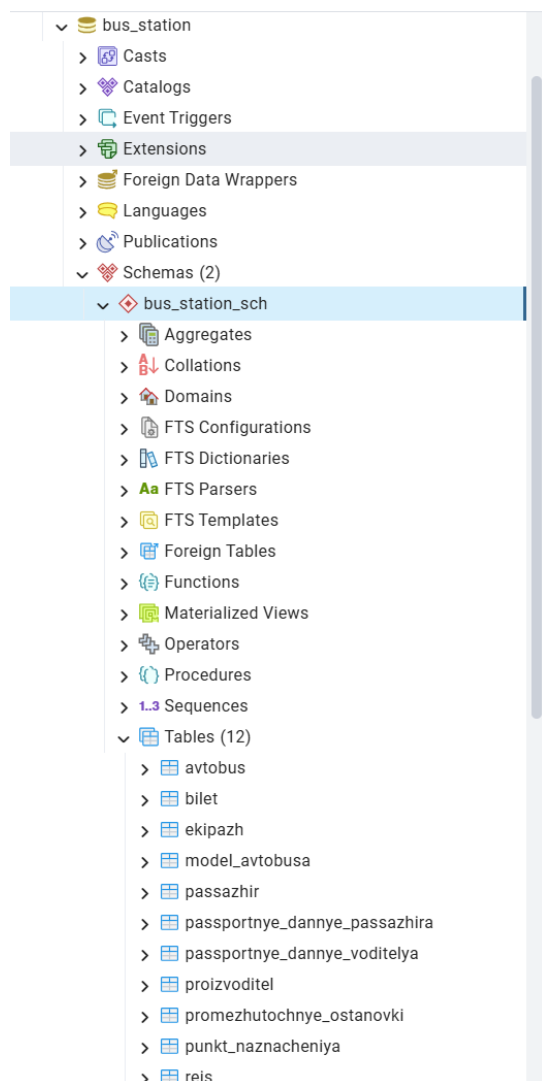
На основе инфологической модели были созданы таблицы, описывающие основные сущности предметной области: производитель, модель автобуса, автобус, пункт назначения, водитель, пассажир, паспортные данные водителя, паспортные данные пассажира, рейс, промежуточные остановки, экипаж и билет.

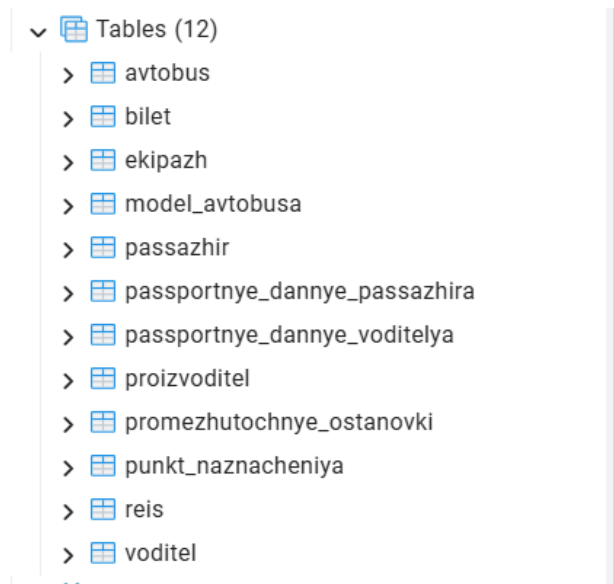
Для обеспечения целостности данных были заданы ограничения следующих типов:

- Primary Key - для идентификации строк таблиц;
- Foreign Key - для обеспечения ссылочной целостности между связанными таблицами;
- Check - для контроля допустимых значений атрибутов;
- Unique - для исключения дублирования отдельных сочетаний значений.

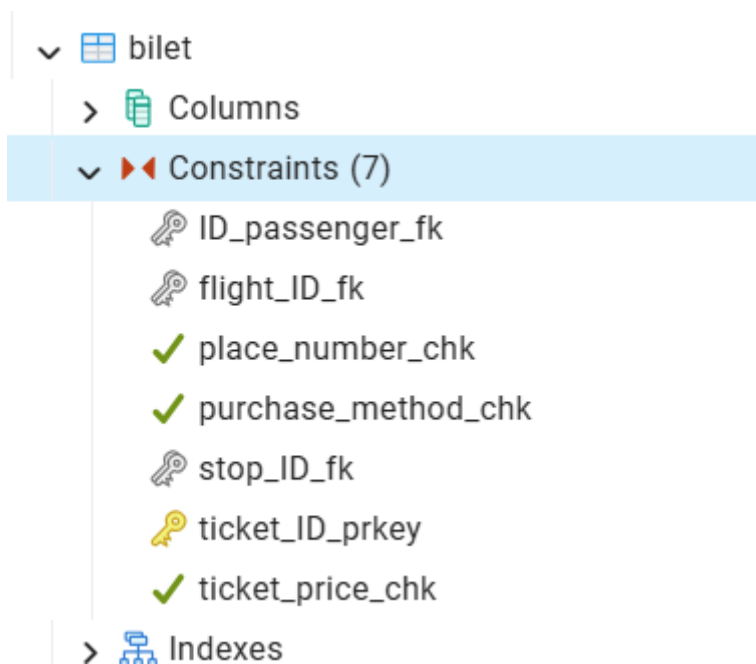
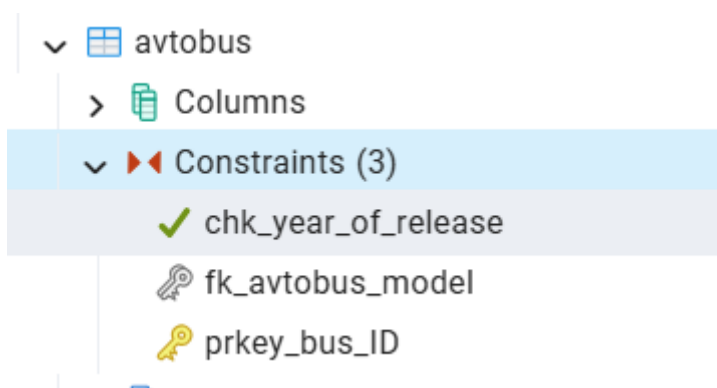
После создания структуры базы данных таблицы были заполнены рабочими данными с использованием Query Tool и SQL-команд INSERT. Корректность заполнения проверялась с помощью запросов вида `SELECT * FROM <имя_таблицы>`

Скрин базы данных:





Таблицы:



▼ ekipazh

> Columns

▼ Constraints (6)

-  ID_crew_prkey
-  driver_ID_fk
- ✓ driver_role_chk
-  flight_ID_fk
- ✓ medical_check_up_status_chk
- ✓ reason_non_admission_chk

▼ Indexes

▼ model_avtobusa

> Columns

▼ Constraints (4)

- ✓ chk_model_bus_type
- ✓ chk_model_place_number
-  fk_model_manufacture
-  model_avtobusa_pkey

> Indexes

▼ passazhir

> Columns

▼ Constraints (2)

-  ID_passenger_prkey
-  ID_passport_pas_fk

▼ Indexes

▼ passportnye_dannye_passazhira

> Columns

▼ Constraints (2)

-  ID_passport_pas_prkey
- ① uq_passenger_passport_batch_number

> Indexes

▼ passportnye_dannye_voditelya

> Columns

▼ Constraints (2)

🔑 ID_passport_prkey

① uq_driver_passport_batch

> Indexes

▼ proizvoditel

> Columns

▼ Constraints (1)

🔑 proizvoditel_pkey

> Indexes

▼ promezhutochnye_ostanovki

> Columns

▼ Constraints (6)

🔑 destination_ID_fk

🔑 flight_ID_fk

🔑 stop_ID_prkey

✓ stop_number_chk

✓ stop_time_chk

① uq_stop_flight_stopnumber

> Indexes

▼ punkt_naznacheniya

> Columns

▼ Constraints (2)

✓ chk_destination_type

🔑 destination_ID_prkey

> Indexes

▼  reis

>  Columns

▼  Constraints (4)

 bus_ID_fk

 destination_ID_fk

 flight_ID_prkey

 flight_type_chk

▼  voditel

>  Columns

▼  Constraints (2)

 ID_passport_fk

 driver_ID_prkey

История запросов:

bus_station/postgres@PostgreSQL 15

No limit

QueryQuery History

Show queries generated internally by pgAdmin?

Remove

Remove All

26.03.2026

22:46:09

▶ INSERT INTO bus_station_sch.bilet ("ticket_ID", place_n...

22:45:43

▶ INSERT INTO bus_station_sch.bilet ("ticket_ID", place_n...

22:45:27

▶ INSERT INTO bus_station_sch.bilet (ticket_ID, place_num...

22:41:04

▶ SELECT * FROM bus_station_sch.ekipazh

22:40:40

▶ INSERT INTO bus_station_sch.ekipazh ("ID_crew", "flight...

22:36:37

▶ SELECT * FROM bus_station_sch.promezhutochnye_ostanovki

22:36:03

▶ INSERT INTO bus_station_sch.promezhutochnye_ostanovki (...

22:32:02

▶ SELECT * FROM bus_station_sch.reis;

22:31:25

▶ INSERT INTO bus_station_sch.reis("flight_ID", departure...

22:28:59

▶ SELECT * FROM bus_station_sch.passazhir;

22:28:36

▶ INSERT INTO bus_station_sch.passazhir("ID_passenger", l...

22:28:14

▶ INSERT INTO bus_station_sch.passazhir(ID_passenger, las...

22:26:51

▶ SELECT * FROM bus_station_sch.voditel;

Total rows:

27.03.2

Date

Copy

SELECT

Message

Success

Query Query History

Show queries generated internally by pgAdmin? ☒

Remove

Remove All

27.03.2026

▶ SELECT * FROM bus_station_sch.voditel;

11:16:31

26.03.2026

▶ SELECT batch, number, COUNT(*) FROM bus_station_sch.pas...

22:58:52

▶ SELECT batch, number, COUNT(*) FROM bus_station_sch.pas...

22:58:34

▶ SELECT batch, number, COUNT(*) FROM bus_station_sch.pas...

22:58:20

▶ SELECT * FROM bus_station_sch.promezhutochnye_ostanovki;

22:47:21

▶ SELECT * FROM bus_station_sch.pomezhutochnye_ostanovki;

22:47:12

▶ SELECT * FROM bus_station_sch.bilet;

22:46:30

▶ INSERT INTO bus_station_sch.bilet ("batch", "number", "date", "time", "status", "price") VALUES ('2026-03-26', '11:16:31', '2026-03-26', '11:16:31', 'active', 100);

Пример команды вставки данных:

QueryQuery History

Show queries generated internally by pgAdmin?☒

Remove

Remove All

27.03.2026

▶ SELECT * FROM bus_station_sch.voditel;
11:16:31

26.03.2026

▶ SELECT batch, number, COUNT(*) FROM bus_sta...
22:58:52

▶ SELECT batch, number, COUNT(*) FROM bus_sta...
22:58:34

▶ SELECT batch, number, COUNT(*) FROM bus_sta...
22:58:20

▶ SELECT * FROM bus_station_sch.promezhutochn...
22:47:21

▶ SELECT * FROM bus_station_sch.pomezhtochny...
22:47:12

▶ SELECT * FROM bus_station_sch.bilet;
22:46:30

▶ INSERT INTO bus_station_sch.bilet ("ticket_...
22:46:09

▶ INSERT INTO bus_station_sch.bilet ("ticket_...
22:45:43

▶ INSERT INTO bus_station_sch.bilet ("ticket_...

26.03.2026 22:46:09

179 msec

DateRows affectedDuration

Copy

Copy to Query Editor

INSERT INTO bus_station_sch.bilet
("ticket_ID", place_number, purchase_date, ticke
VALUES
(1, 12, '2025-04-05', 1500.00, 'K', 1, 1, 1),
(2, 18, '2025-04-07', 1500.00, 'O', 1, 2, 2),
(3, NULL, '2025-04-08', 1080.00, 'O', 2, 3, 4);

Messages

Query returned successfully in 179 msec.

INSERT INTO bus_station_sch.bilet

("ticket_ID", place_number, purchase_date, ticket_price, purchase_method, "flight_ID",
"ID_passenger", "stop_ID")

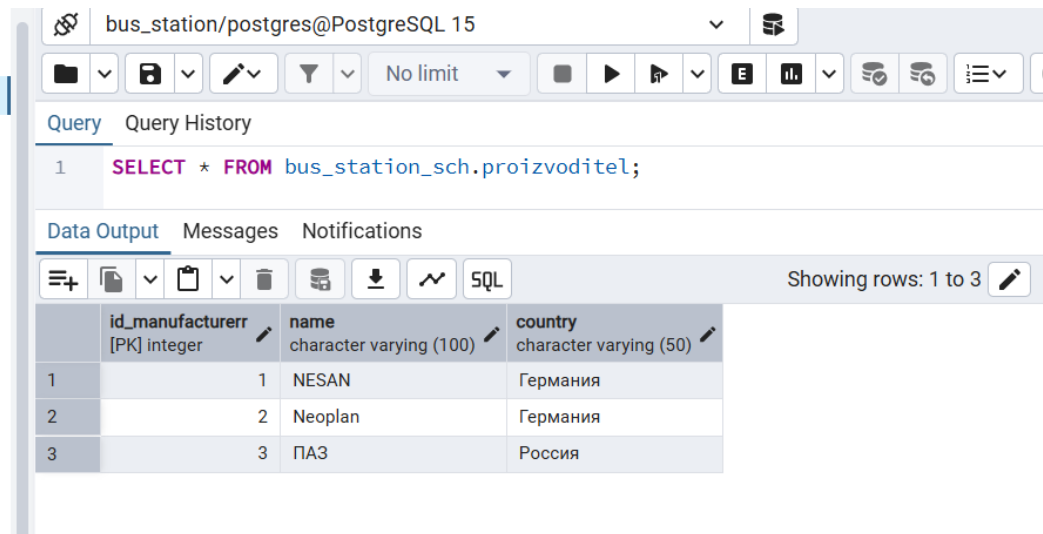
VALUES

(1, 12, '2025-04-05', 1500.00, 'K', 1, 1, 1),

(2, 18, '2025-04-07', 1500.00, 'O', 1, 2, 2),

(3, NULL, '2025-04-08', 1080.00, 'O', 2, 3, 4);

Пример заполненных данных:



The screenshot shows a PostgreSQL client interface with the following details:

- Connection: bus_station/postgres@PostgreSQL 15
- Query: `SELECT * FROM bus_station_sch.proizvoditel;`
- Data Output table with 3 rows:

	id_manufacturerr [PK] integer	name character varying (100)	country character varying (50)
1	1	NESAN	Германия
2	2	Neoplan	Германия
3	3	ПАЗ	Россия

Раздел про резервное копирование:

Резервная копия создавалась с использованием диалога Backup..., вызываемого из контекстного меню базы данных. Сделанные Backup для формата Custom и Backup для формата Plain.

Имя файла	Дата резервного копирования	Тип резервного копирования	Размер
 bus_station_custom.backup	26.03.2026 23:37	Файл "BACKUP"	24 КБ
 bus_station_dump.sql	26.03.2026 23:46	Исходный файл SQL	23 КБ

dump, содержащий скрипты работы с БД:

Создание базы данных и схемы:

```
CREATE DATABASE bus_station WITH TEMPLATE = template0 ENCODING = 'UTF8' LOCALE_PROVIDER = libc LOCALE = 'Russian_Russia.1251';  
  
ALTER DATABASE bus_station OWNER TO postgres;
```

```
CREATE SCHEMA bus_station_sch;  
  
ALTER SCHEMA bus_station_sch OWNER TO postgres;
```

Пример создания таблиц:

```
5
6 CREATE TABLE bus_station_sch.avtobus (
7     "bus_ID" character(10) NOT NULL,
8     year_of_release integer NOT NULL,
9     "model_ID" integer NOT NULL,
10    CONSTRAINT chk_year_of_release CHECK ((year_of_release > 1980))
11 );
12
13
14 ALTER TABLE bus_station_sch.avtobus OWNER TO postgres;
15
16 --
17 -- TOC entry 226 (class 1259 OID 16521)
18 -- Name: bilet; Type: TABLE; Schema: bus_station_sch; Owner: postgres
19 --
20
21 CREATE TABLE bus_station_sch.bilet (
22     "ticket_ID" integer NOT NULL,
23     place_number integer,
24     purchase_date date NOT NULL,
25     ticket_price numeric NOT NULL,
26     purchase_method character(1) NOT NULL,
27     "flight_ID" integer NOT NULL,
28     "ID_passenger" integer NOT NULL,
29     "stop_ID" integer NOT NULL,
30     CONSTRAINT place_number_chk CHECK (((place_number IS NULL) OR (place_number > 0))),
31     CONSTRAINT purchase_method_chk CHECK ((purchase_method = ANY (ARRAY['K'::bpchar, 'O'::bpchar]))),
32     CONSTRAINT ticket_price_chk CHECK ((ticket_price > (0)::numeric))
33 );
34
35 ALTER TABLE bus_station_sch.bilet OWNER TO postgres;
36
37 --
38
```

Пример команд заполнения таблиц данными:

```
INSERT INTO bus_station_sch.avtobus VALUES ('BUS001', 2018, 1);
INSERT INTO bus_station_sch.avtobus VALUES ('BUS002', 2020, 2);
INSERT INTO bus_station_sch.avtobus VALUES ('BUS003', 2017, 3);

--
-- TOC entry 3426 (class 0 OID 16521)
-- Dependencies: 226
-- Data for Name: bilet; Type: TABLE DATA; Schema: bus_station_sch; Owner: postgres
--

INSERT INTO bus_station_sch.bilet VALUES (1, 12, '2025-04-05', 1500.00, 'K', 1, 1, 1);
INSERT INTO bus_station_sch.bilet VALUES (2, 18, '2025-04-07', 1500.00, 'O', 1, 2, 2);
INSERT INTO bus_station_sch.bilet VALUES (3, NULL, '2025-04-08', 1080.00, 'O', 2, 3, 4);

--
-- TOC entry 3425 (class 0 OID 16503)
-- Dependencies: 225
-- Data for Name: ekipazh; Type: TABLE DATA; Schema: bus_station_sch; Owner: postgres
--

INSERT INTO bus_station_sch.ekipazh VALUES (1, 1, 1, 'первый водитель', '2025-04-10', 'P', NULL);
INSERT INTO bus_station_sch.ekipazh VALUES (2, 1, 2, 'второй водитель', '2025-04-10', 'P', NULL);
INSERT INTO bus_station_sch.ekipazh VALUES (3, 2, 1, 'первый водитель', '2025-04-10', 'P', NULL);
INSERT INTO bus_station_sch.ekipazh VALUES (4, 2, 2, 'второй водитель', '2025-04-10', 'N', 'Повышенное давление');
```

Пример установки primary key:

```
ALTER TABLE ONLY bus_station_sch.ekipazh
|   ADD CONSTRAINT "ID_crew_prkey" PRIMARY KEY ("ID_crew");

--
-- TOC entry 3249 (class 2606 OID 16464)
-- Name: passazhir ID_passenger_prkey; Type: CONSTRAINT; Schema: bus_station_sch; Owner: postgres
--

ALTER TABLE ONLY bus_station_sch.passazhir
|   ADD CONSTRAINT "ID_passenger_prkey" PRIMARY KEY ("ID_passenger");

--
-- TOC entry 3243 (class 2606 OID 16449)
-- Name: passportnye_dannye_passazhira ID_passport_pas_prkey; Type: CONSTRAINT; Schema: bus_station_sch; Owner: postgres
--

ALTER TABLE ONLY bus_station_sch.passportnye_dannye_passazhira
|   ADD CONSTRAINT "ID_passport_pas_prkey" PRIMARY KEY ("ID_passport");

--
```

Пример наложения unique:

```
ALTER TABLE ONLY bus_station_sch.passportnye_dannye_voditelya
|   ADD CONSTRAINT uq_driver_passport_batch UNIQUE (batch, number);

--
-- TOC entry 3245 (class 2606 OID 16549)
-- Name: passportnye_dannye_passazhira uq_passenger_passport_batch_number; Type: CONSTRAINT; Schema: bus_station_sch; Owner: postgres
--

ALTER TABLE ONLY bus_station_sch.passportnye_dannye_passazhira
|   ADD CONSTRAINT uq_passenger_passport_batch_number UNIQUE (batch, number);

--
-- TOC entry 3255 (class 2606 OID 16551)
-- Name: promezhutochnye_ostanovki uq_stop_flight_stopnumber; Type: CONSTRAINT; Schema: bus_station_sch; Owner: postgres
--

ALTER TABLE ONLY bus_station_sch.promezhutochnye_ostanovki
|   ADD CONSTRAINT uq_stop_flight_stopnumber UNIQUE ("flight_ID", stop_number);

--
-- TOC entry 3270 (class 2606 OID 16536)
-- Name: билет ID_passenger_fk; Type: FK CONSTRAINT; Schema: bus_station_sch; Owner: postgres
--

ALTER TABLE ONLY bus_station_sch.билет
|   ADD CONSTRAINT "ID_passenger_fk" FOREIGN KEY ("ID_passenger") REFERENCES bus_station_sch.passazhir("ID_passenger");
```

Пример наложения ограничений на данных:

```
CREATE TABLE bus_station_sch.punkt_naznacheniya (
|   "destination_ID" integer NOT NULL,
|   destination_name character varying(100) NOT NULL,
|   destination_type character varying(30) NOT NULL,
|   CONSTRAINT chk_destination_type CHECK (((destination_type)::text = ANY ((ARRAY['страна'::character varying, 'город'::character varying, 'посёлок'::character varying, 'деревня'::character varying])::text)))
|);

ALTER TABLE bus_station_sch.punkt_naznacheniya OWNER TO postgres;

--
-- TOC entry 223 (class 1259 OID 16470)
-- Name: reis; Type: TABLE; Schema: bus_station_sch; Owner: postgres
--

CREATE TABLE bus_station_sch.reis (
|   "flight_ID" integer NOT NULL,
|   departure_date date NOT NULL,
|   departure_time time without time zone NOT NULL,
|   flight_type character(1) NOT NULL,
|   "bus_ID" character(10) NOT NULL,
|   "destination_ID" integer NOT NULL,
|   CONSTRAINT flight_type_chk CHECK ((flight_type = ANY (ARRAY['R'::bpchar, 'D'::bpchar])))
|);
```

Пример установки foreign key:

```
ALTER TABLE ONLY bus_station_sch.voditel
|   ADD CONSTRAINT "ID_passport_fk" FOREIGN KEY ("ID_passport") REFERENCES bus_station_sch.passportnye_dannye_voditelya("ID_passport");

--
-- TOC entry 3263 (class 2606 OID 16465)
-- Name: passazhir ID_passport_pas_fk; Type: FK CONSTRAINT; Schema: bus_station_sch; Owner: postgres
--

ALTER TABLE ONLY bus_station_sch.passazhir
|   ADD CONSTRAINT "ID_passport_pas_fk" FOREIGN KEY ("ID_passport") REFERENCES bus_station_sch.passportnye_dannye_passazhira("ID_passport");

--
-- TOC entry 3264 (class 2606 OID 16476)
-- Name: reis bus_ID_fk; Type: FK CONSTRAINT; Schema: bus_station_sch; Owner: postgres
--

ALTER TABLE ONLY bus_station_sch.reis
|   ADD CONSTRAINT "bus_ID_fk" FOREIGN KEY ("bus_ID") REFERENCES bus_station_sch.avtobus("bus_ID");

--
-- TOC entry 3266 (class 2606 OID 16498)
-- Name: promezhutochnye_ostanovki destination_ID_fk; Type: FK CONSTRAINT; Schema: bus_station_sch; Owner: postgres
--

ALTER TABLE ONLY bus_station_sch.promezhutochnye_ostanovki
|   ADD CONSTRAINT "destination_ID_fk" FOREIGN KEY ("destination_ID") REFERENCES bus_station_sch.punkt_naznacheniya("destination_ID");
```

Восстановление базы данных:

Для проверки корректности созданной резервной копии было выполнено восстановление базы данных средствами pgAdmin 4 с использованием диалога Restore.... В качестве источника восстановления использовалась резервная копия в формате Custom.

- ✓  PostgreSQL 15
 - ✓  Databases (3)
 - >  bus_station
 - ✓  bus_station_restore
 - >  Casts
 - >  Catalogs
 - >  Event Triggers
 - >  Extensions
 - >  Foreign Data Wrappers
 - >  Languages
 - >  Publications
 - ✓  Schemas (2)
 - >  bus_station_sch
 - >  public
 - >  Subscriptions
 - >  postgres
 - >  Login/Group Roles

- ▼  bus_station_restore
 - >  Casts
 - >  Catalogs
 - >  Event Triggers
 - >  Extensions
 - >  Foreign Data Wrappers
 - >  Languages
 - >  Publications
 - ▼  Schemas (2)
 - ▼  bus_station_sch
 - >  Aggregates
 - >  Collations
 - >  Domains
 - >  FTS Configurations
 - >  FTS Dictionaries
 - >  **Aa** FTS Parsers
 - >  FTS Templates
 - >  Foreign Tables
 - >  Functions
 - >  Materialized Views
 - >  Operators
 - >  Procedures
 - >  **1..3** Sequences
 - ▼  Tables (12)
 - >  avtobus
 - >  bilet
 - >  ekipazh
 - >  model_avtobusa
 - >  passazhir
 - >  passportnye_dannye_passazhira
 - >  passportnye_dannye_voditelya
 - >  proizvoditel
 - >  promezhutochnye_ostanovki
 - >  punkt_naznacheniya
 - >  reis

Поскольку исходная рабочая база данных уже существовала и использовалась для выполнения лабораторной работы, восстановление было выполнено в предварительно созданную пустую базу данных. После выполнения процедуры восстановления в новой базе были успешно восстановлены структура схемы, таблицы, ограничения и рабочие данные, что подтверждает корректность созданной резервной копии.

Выводы

В ходе выполнения лабораторной работы были получены практические навыки создания базы данных в PostgreSQL с использованием pgAdmin 4. Была реализована база данных по индивидуальному варианту «Автовокзал»: создана схема, созданы таблицы, заданы ограничения Primary Key, Foreign Key, Check и Unique, таблицы заполнены рабочими данными.

Также были освоены средства резервного копирования и восстановления базы данных. Были созданы две резервные копии: архивная в формате Custom и текстовая в формате Plain. Выполненное восстановление базы данных подтвердило корректность сформированной структуры БД и сохраненных данных.

Список источников

- Документация PostgreSQL [Электронный ресурс] // Официальный сайт PostgreSQL. 1996-2025.
- Документация pgAdmin 4 PostgreSQL [Электронный ресурс] // Официальный сайт pgAdmin.